

HARDWARE ACCELERATION OF CONTAINER NETWORKING INTERFACES

Aug 2020, Netdev 0x14

Amritha Nambiar

Sridhar Samudrala

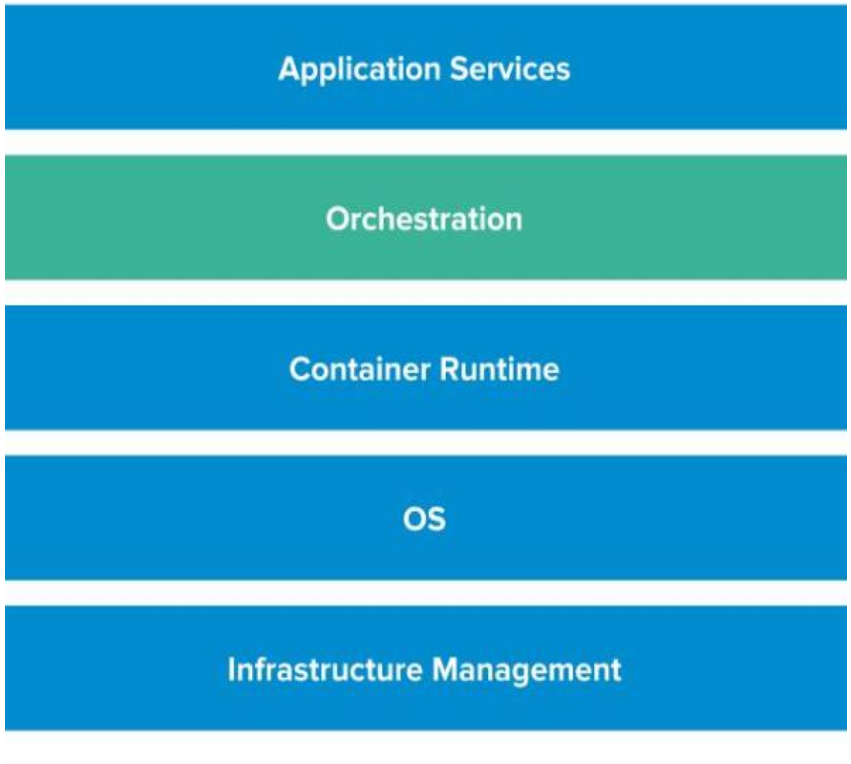
Kiran Patil

Agenda

- Linux Virtual Networking for containers
- Problem statement
- Why is Hardware acceleration needed ?
- Container networking models
  - MACVLAN
  - IPVLAN
  - Subfunction
- Control and Configurability
- Next Steps
- Summary



Container Networking Landscape



Credit: Mike Kavis



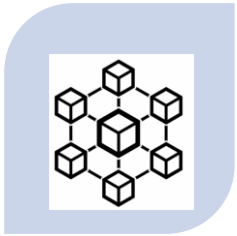
Credit: Shuo

Linux Virtual Networking for containers

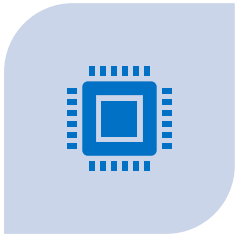
Linux virtual networking interfaces:

- Stacked virtual network devices on top of a physical PCIe PF netdev device
- Assigned to container namespaces and share some of the PCI device resources
- Common examples currently used in deployments: macvlan, ipvlan
- SR-IOV VFs as container interfaces have limitations:
  - Scalability - Container systems have a lower overhead than VMs and container systems typically target environments where thousands of containers are in play
  - Separate PCI function per VF, separate config space
  - HW based packet replication for broadcast, multicast – higher PCI BW utilization

Problem statement



In real CSP deployments, large number of containers are spawned on a server



Network devices are becoming faster (100/200/400 Gbps), programmable, and support 1000’s of HW queues. These devices provide isolable and selectable entities (Queue Groups)



Effective utilization of the system resources (cores, memory, network) is required to improve performance of containers



This calls for a mechanism to accelerate virtual networking interfaces and provide HW level resource isolation

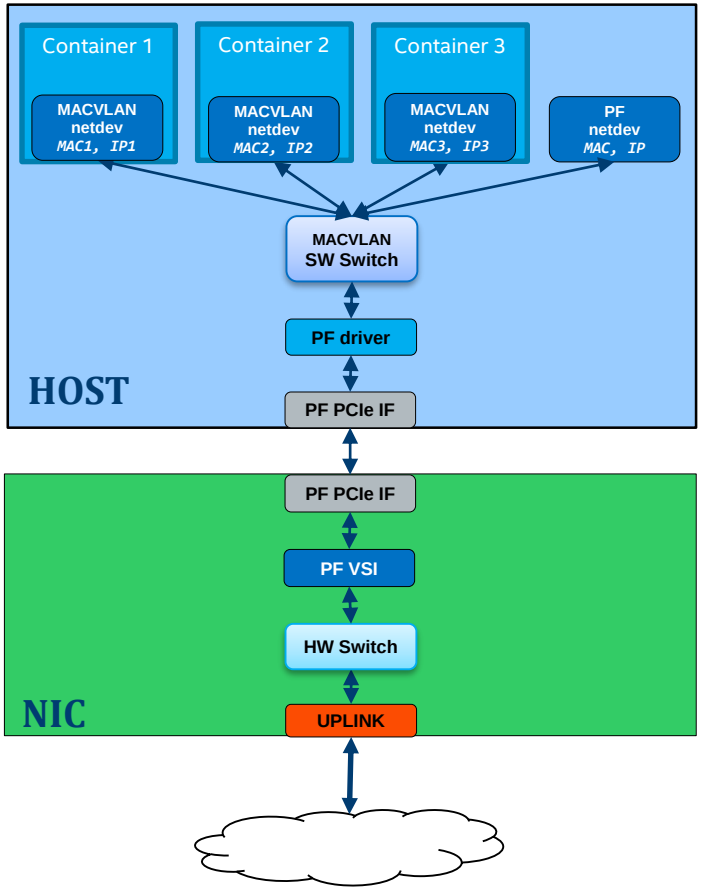
Why is Hardware Acceleration Needed ?

- Dedicate hardware device resources to container applications :
  - Make it possible to split traffic over multiple CPUs through isolation
  - No synchronization overhead in the fast path
- Avoid expensive lookup operations in the receive path for unicast traffic
- Allow control and configurability of HW offload parameters
- Support multiple use cases such as containers, VMs, applications running in bare-metal environment

How to Accelerate Containers

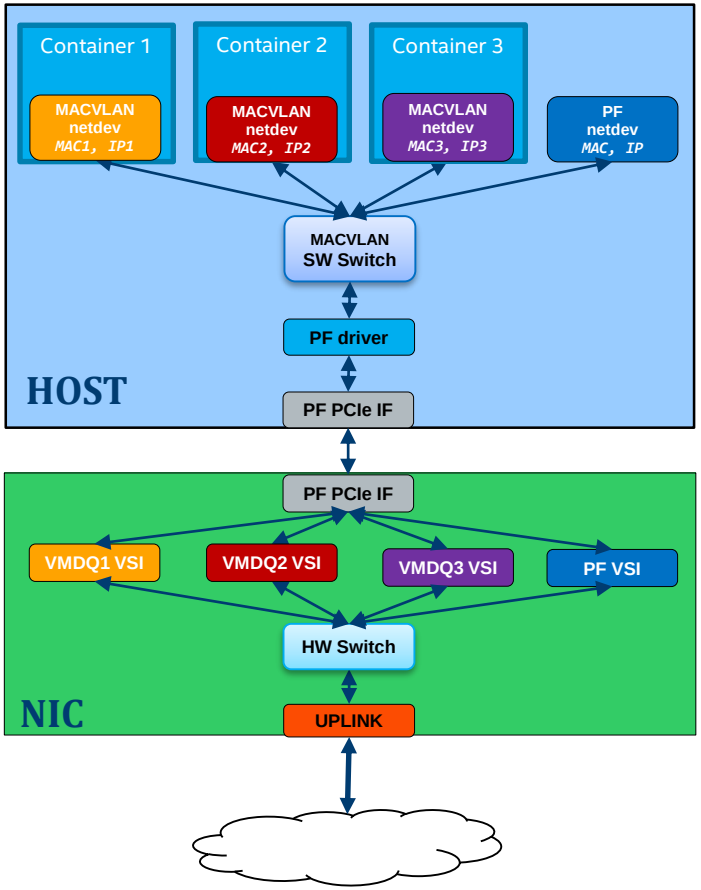
- Create queue-sets or VSIs with dedicated set of resources
  - TX/RX Queues, Interrupt Vectors, RSS Lookup Table
- Assign these queue-sets to virtual interfaces in containers
- Add HW Switch filters to direct traffic to these queue-sets
- VMDq is one such technology that enables creating these queue-sets
- Stacked Networking Interfaces
  - Acceleration supported for MACVLAN interfaces on Intel NICs
  - Acceleration can be enabled for IPVLAN interfaces
- Native networking Interfaces
  - Subfunctions

MACVLAN



- Shared HW resources
- DMAC rules in HW Switch
  - All DMACs -> PF VSI
- DMAC lookup in SW Switch
  - Select the right netdev
- Separate MAC address
- Separate IP address
- East West traffic via SW Switch

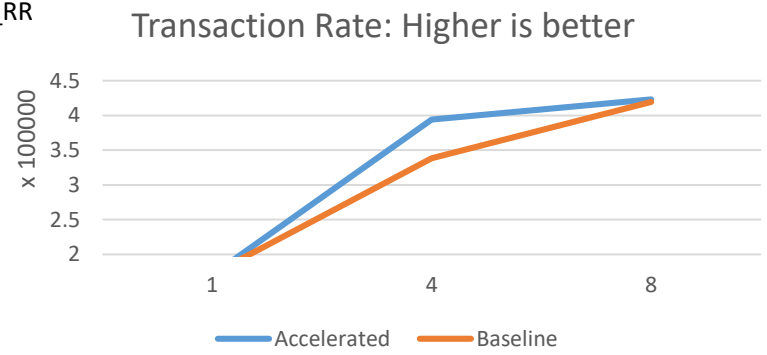
MACVLAN Accelerated



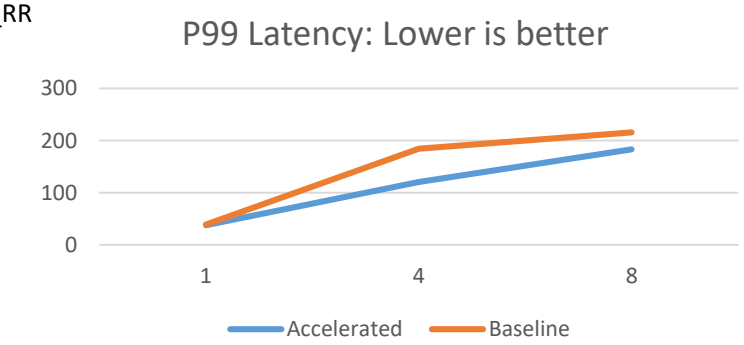
- **Dedicated HW resources**
- DMAC rules in HW Switch
  - **Per VSI : DMAC -> VSI**
- **No DMAC lookup in SW Switch**
  - **Driver selects the right netdev**
- Separate MAC address
- Separate IP address
- East West traffic via SW Switch

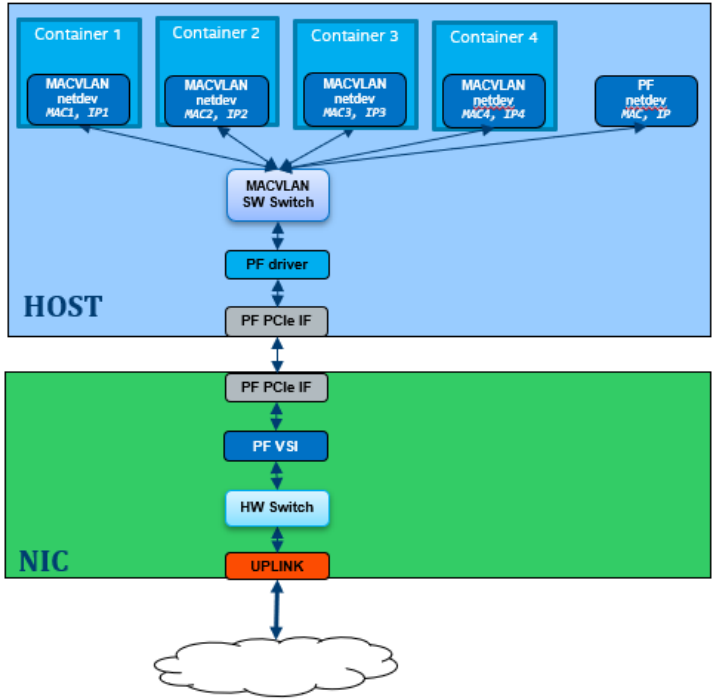
Performance

TCP_RR



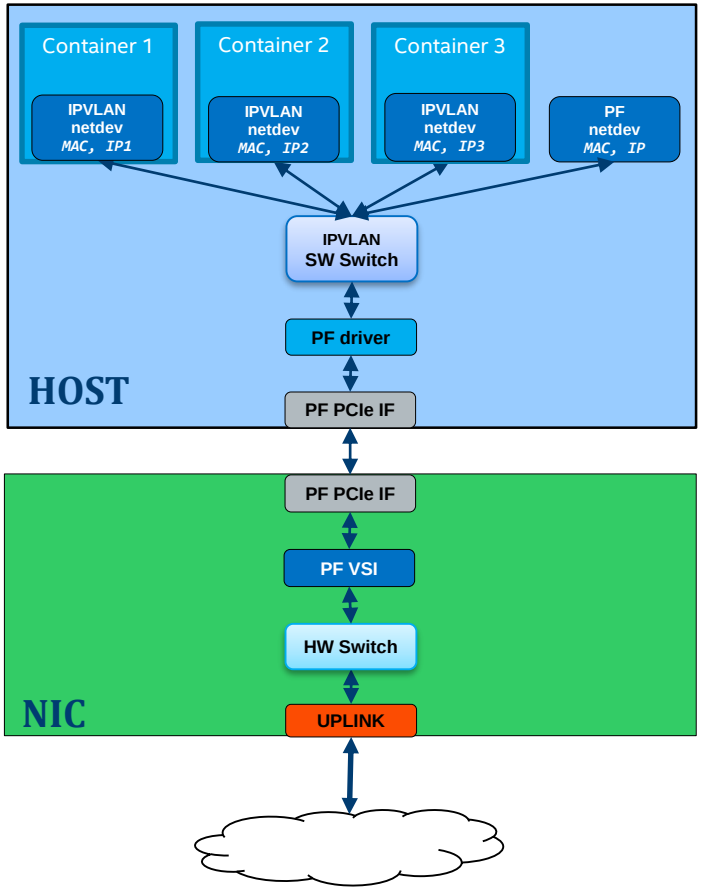
TCP_RR





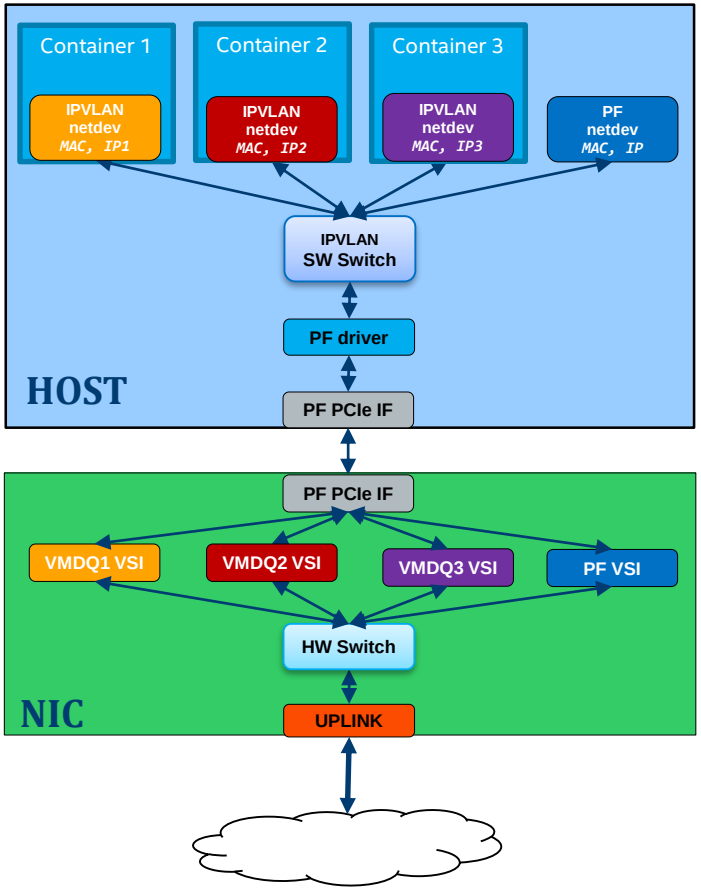
MACVLAN Setup

IPVLAN



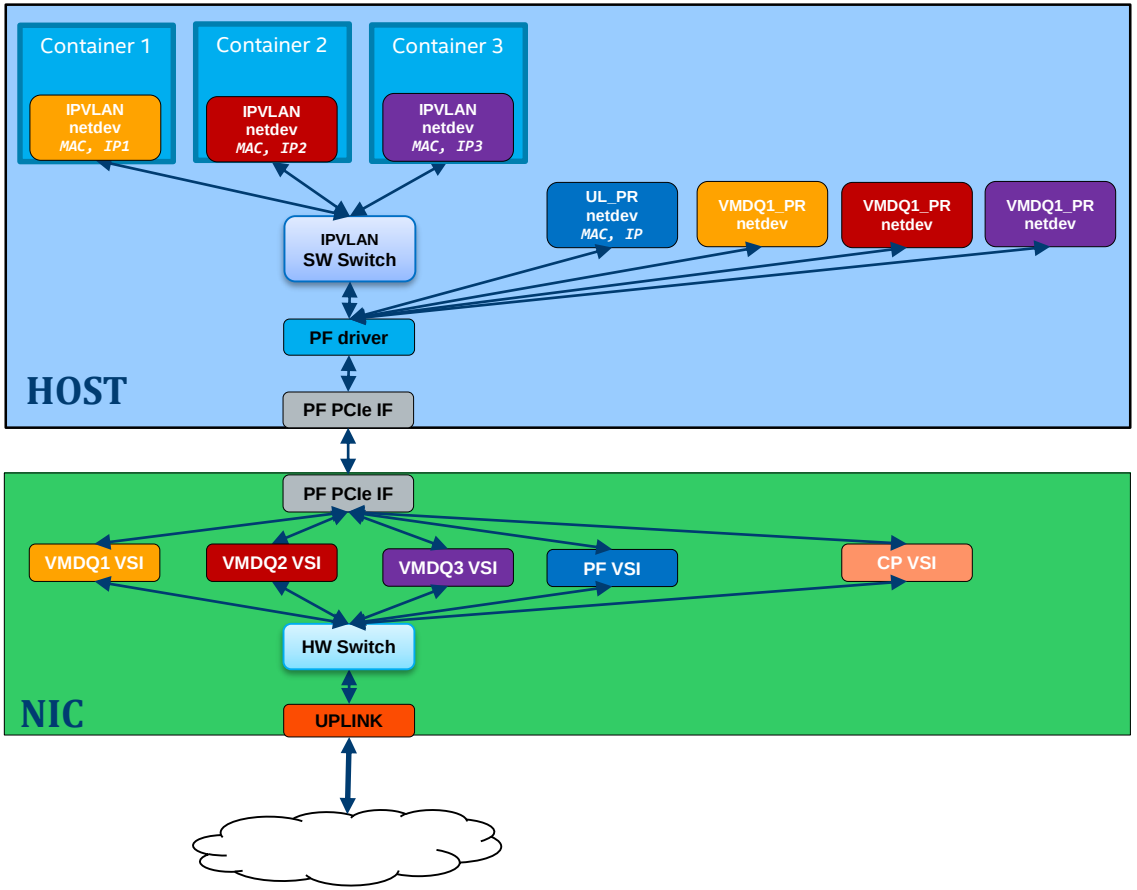
- Shared HW resources
- DMAC rule in HW Switch
  - DMAC -> PF VSI
- DIP lookup in SW Switch
  - Select the right netdev
- Shared MAC address
- Separate IP address
- East West traffic via SW Switch

IPVLAN Accelerated



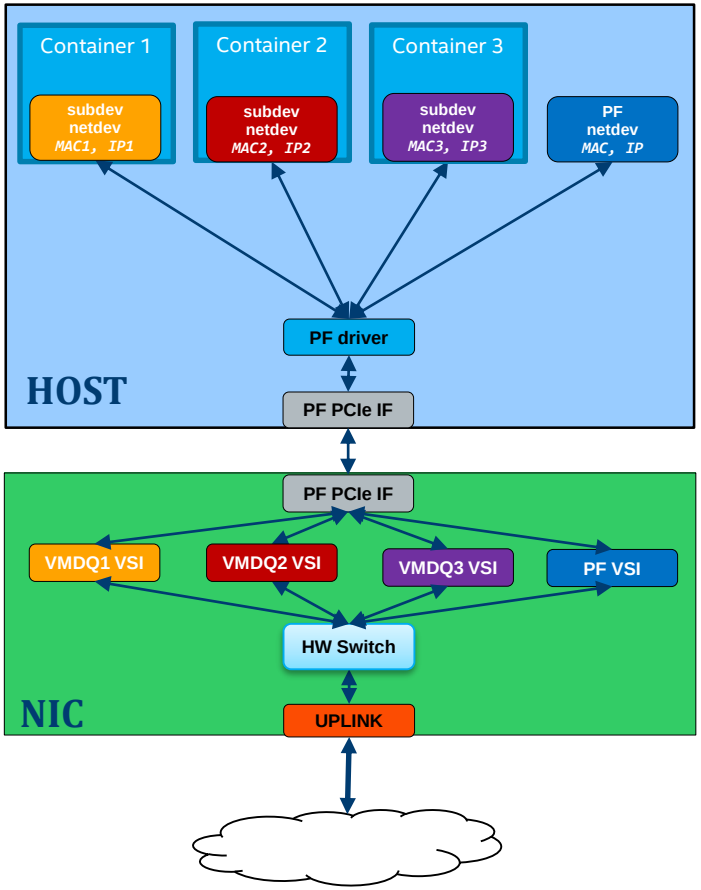
- Dedicated HW resources
- DMAC + DIP rules in HW Switch
  - DMAC -> PF VSI
  - DMAC + DIP -> VMDQ VSI
- No IP lookup in SW Switch
  - Driver selects the right netdev
- Shared MAC address
- Separate IP address
- East West traffic via SW Switch

IPVLAN Accelerated – Switchdev Mode



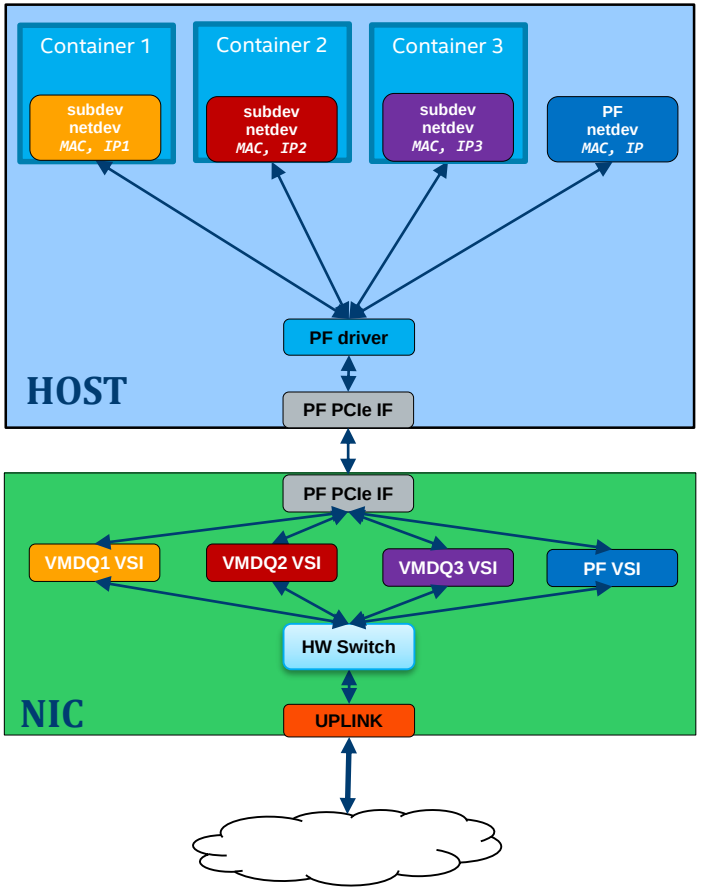
- Dedicated HW resources
- Control Plane adds rules to HW Switch
  - L2, L3, L4 and tunnel matches
  - encap/decap, push/pop, modify actions
- No IP lookup in SW Switch
  - Driver selects the right netdev
- Shared MAC address
- Separate IP address
- East West traffic via SW Switch

Subfunction : CDQ (L2 mode)



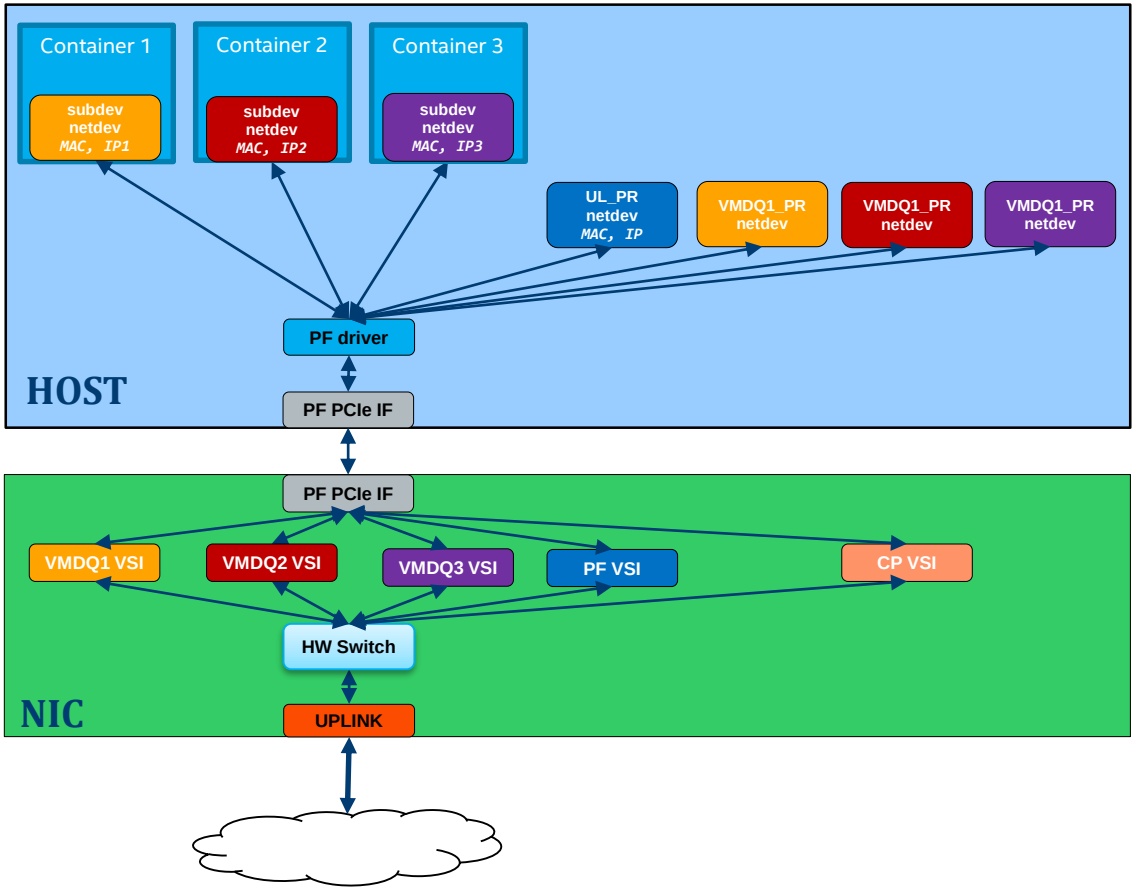
- Dedicated HW resources
- DMAC rules in HW Switch
  - Per VSI : DMAC -> VSI
- PF driver acts as Software Switch
- Separate MAC address
- Separate IP address
- netdev_ops, ethtool_ops on subfunction device
- XDP, AF_XDP supported
- East West traffic via SW Switch in PF driver

Subfunction : CDQ (L3 mode)



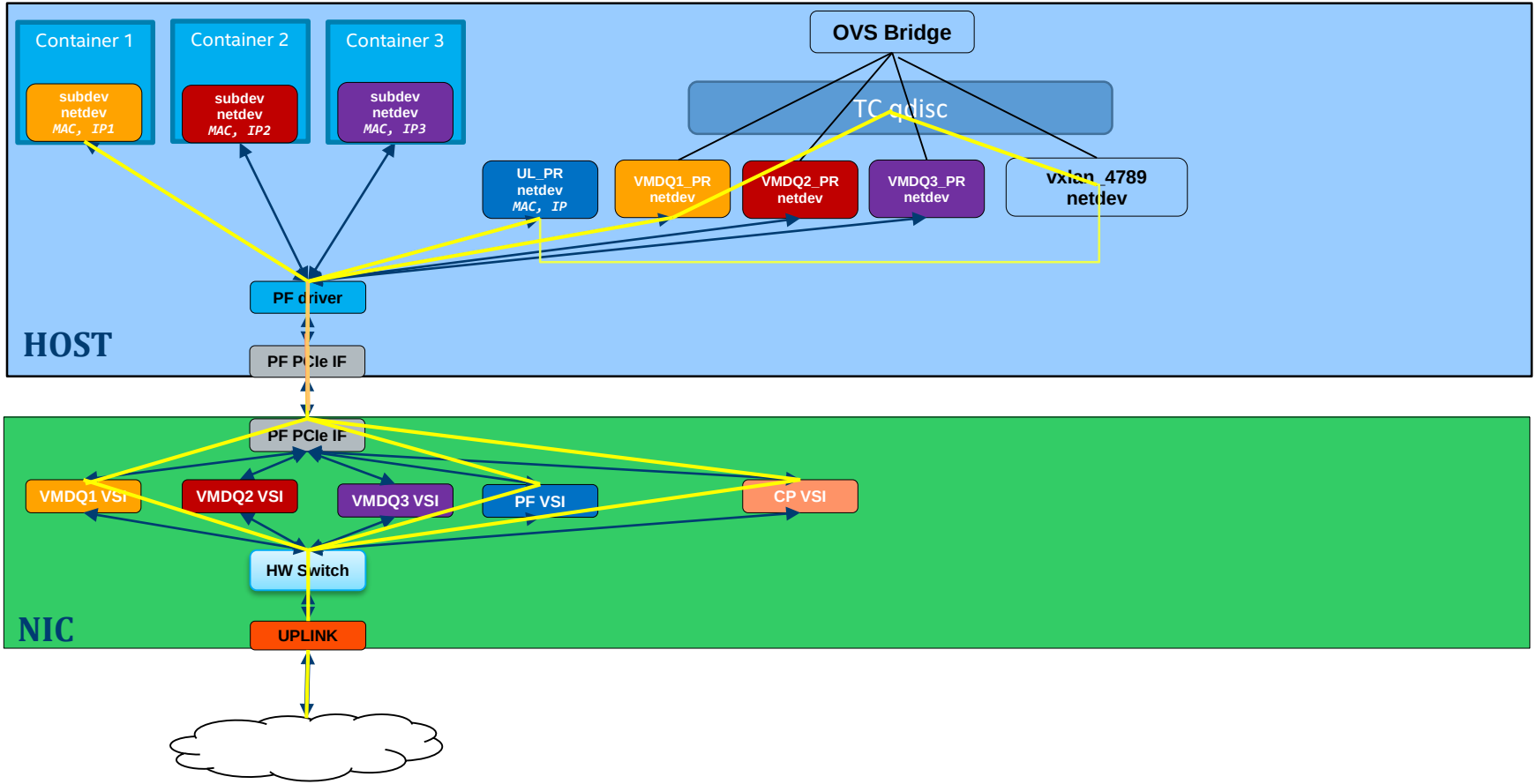
- Dedicated HW resources
- DMAC rules in HW Switch
  - Per VSI : DMAC -> VSI
- PF driver acts as Software Switch
- Same MAC address
- Separate IP address
- netdev_ops, ethtool_ops on subfunction device
- XDP, AF_XDP supported
- East West traffic via SW Switch in PF driver

Subfunction : CDQ – Switchdev Mode

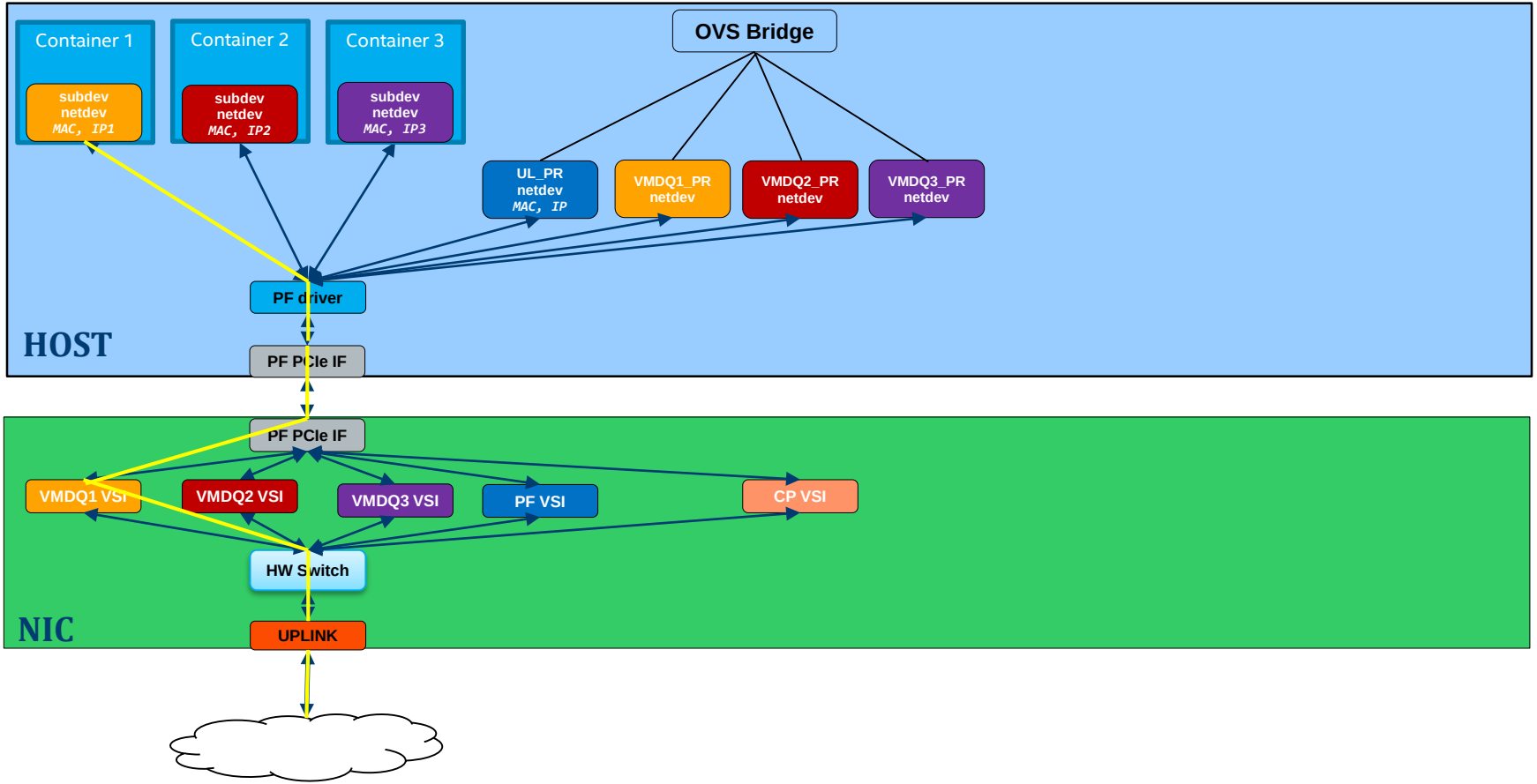


- Dedicated HW resources
- Control Plane adds rules to HW Switch
  - L2, L3, L4 and tunnel matches
  - encap/decap, push/pop, modify actions
- PF driver acts as Software Switch
- Separate/Shared MAC address
- Separate IP address
- netdev_ops, ethtool_ops on subdevice
- XDP, AF_XDP supported
- East West traffic via SW Switch in PF driver

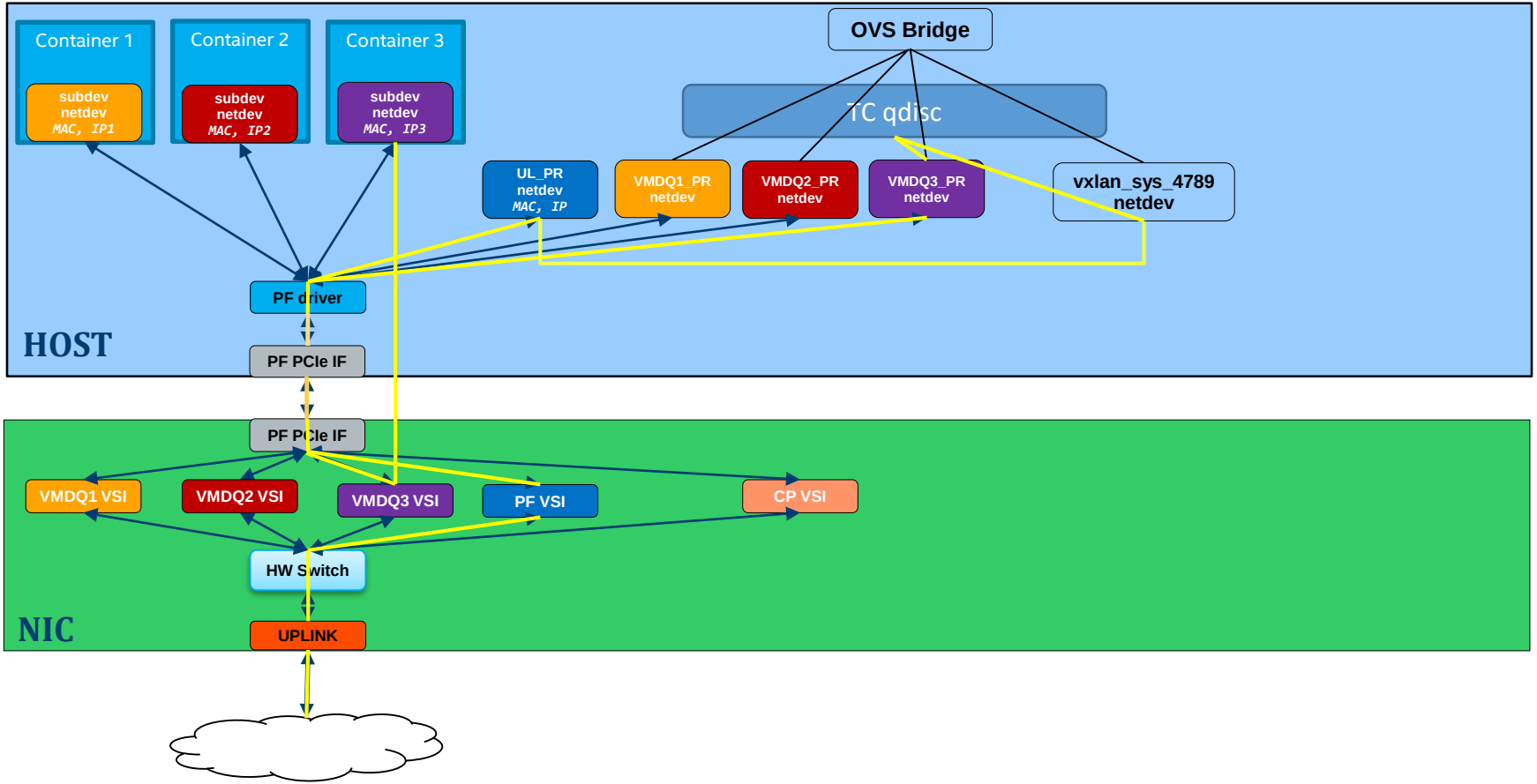
Subfunction Switchdev: SW Assisted Encap



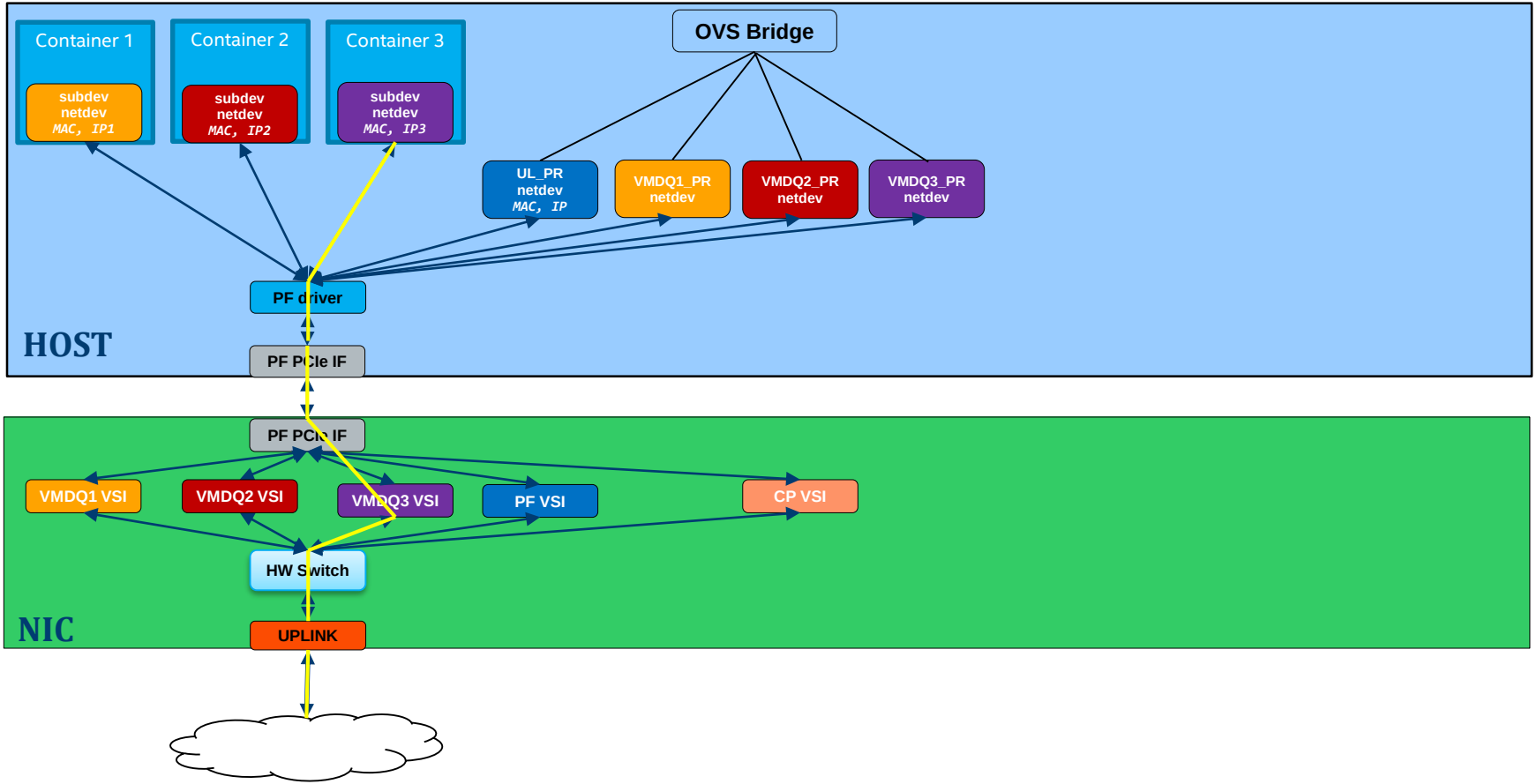
Subfunction Switchdev : HW Encap



Subfunction Switchdev: SW Assisted Decap



Subfunction Switchdev : HW Decap



Control and Configurability

Host Controlled Properties/Features

- Switchdev mode: Accelerated macvlan/ipvlan, subfunction with standard net_device_ops
  - Switch Filters
  – *ndo_setup_tc (port netdev)*
  - Link State
  – *ndo_open/ndo_close* (port netdev)
  - Statistics
  – *ndo_get_offload_stats* (port netdev)
- Devlink extensions on subfunction (SF):
  - Port VLAN
  - TX Rate Limiting
  - Single/Additional MAC

Guest Controlled Properties/Features

- macvlan/ipvlan netdev ops: Limited subset of net_device_ops
  - *ndo_open/ndo_close (macvlan/ipvlan netdev)*
  - *ndo_change_mtu (macvlan netdev)*
  - *ndo_set_mac_address (macvlan netdev)*
  - *ndo_vlan_rx_add_vid (macvlan/ipvlan netdev)*
- macvlan/ipvlan ethtool_ops : Limited
  - *get_link (macvlan/ipvlan netdev)*
  - *get_drv_info(macvlan/ipvlan netdev)*
  - *set/get msglevel (ipvlan netdev)*
- Subfunction netdev_ops/ethtool_ops

Control and Configurability

Policy Configuration on host

- Devlink extensions on subfunction (SF):

- MTU (max)
- Number of Rx/Tx Queues (max)
- Number of Rx/Tx Descriptors (max)
- Rx/Tx Interrupt Moderation (min)
- MSI-X Vectors (max)
- Number of TCs (max)
- Capabilities (on/off)
- Promiscuous mode
- RSS Configuration
- Queue selection filters (FDIR)
- Tagged VLANs

Legacy mode comparison

Feature	macvlan	macvlan accelerated	ipvlan	ipvlan accelerated	Subfunction CDQ
MAC address	Separate	Separate	Shared	Shared	Separate/Shared
HW resources	Shared	Dedicated	Shared	Dedicated	Dedicated
Host controlled properties	No	No	No	No	via devlink extension for some features (Port VLAN, TX rate etc.)
Guest controlled properties	Yes (Limited)	Yes (Limited)	Yes (Limited)	Yes (Limited)	Yes
Policy configuration (Default on PF) from host	No	No	No	No	via devlink extension for get/set parameters on SF
XDP, AF_XDP Support	No	No	No	No	Yes
East West traffic	SW Bridge	SW Bridge	SW Routing	SW Routing	via SW Switch in PF driver
Encap/Decap	In SW if macvlan on vxlan	No support	In SW if ipvlan on vxlan	No Support	No support

Switchdev mode comparison

Feature	macvlan accelerated	ipvlan accelerated	Subfunction CDQ
MAC address	Separate	Shared	Separate/Shared
HW resources	Dedicated	Dedicated	Dedicated
Host controlled properties	Yes	Yes	Yes
Guest controlled properties	Yes (Limited)	Yes (Limited)	Yes
Policy configuration (Default on PF) from host	No	No	via devlink extension for get/set parameters on SF
XDP, AF_XDP Support	No	No	Yes
East West traffic	SW Bridge	SW Routing	via SW Switch in PF driver
Encap/Decap	In HW if capable and also possible in SW via TC rules	In HW if capable and also possible in SW via TC rules	In HW if capable and also possible in SW via TC rules

Next Steps

IPVLAN

Kernel

- Add ndo_*_add/del callback in ipvlan module
- ipvlan_open: offload ipv4/ipv6 addresses
- Address change notifier events for ipv4/ipv6 addresses needs to be relayed down to master

Driver

- PF driver for lower device accelerates ipvlan netdevice as VMDq VSI
- PF driver adds IP based filter (DMAC, DIP, FWD_TO_VSI)

Subfunction CDQ

Kernel

- Devlink creates a subfunction on a given PCI function

Driver

- Binding the subfunction to a driver creates netdev for the subfunction
- Subfunction netdevs accelerated as VMDq VSIs on the host PF driver

Summary



Accelerating virtual networking interfaces for containers improves performance via dedicated HW resources and avoiding expensive SW lookup



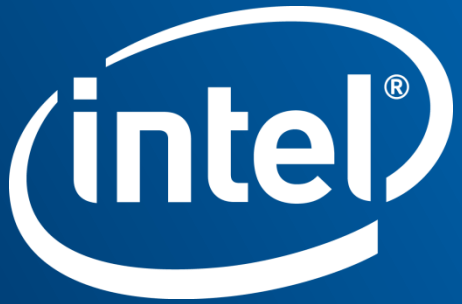
Accelerated virtual networking interfaces in switchdev mode provides improved control and configurability of features



Accelerated models are suitable where north-south traffic is prominent

Acknowledgement

- Anil Vasudevan
- Parthasarathy Sarangam
- Alexander Duyck
- Anjali Singhai Jain
- Jesse Brandeburg


intel®